Project 0 in FMNN10 and NUMN32

© Tony Stillfjord, Monika Eisenmann, Gustaf Söderlind 2026

Goals: The goal of this project is to get started with Python or MATLAB as a tool for approximating solutions to differential equations, and in particular, to get acquainted with time-stepping methods for initial value ODEs, and standard techniques for analyzing and assessing such methods.

Warm-up. This initial "warm-up" project does not have to be presented in any way. But Project 1 is built on the code structures that you develop here, modified for more complicated methods. It is much easier and less total work to figure out any complications in this simpler setting and then transfer that knowledge to the more complicated setting, than trying to do it all at once. Thus, please start here.

Start early. The idea is to start working with these exercises as soon as possible in the first week, identify things you have forgotten or are unsure about, and then get help with these on the first and second exercises.

Contents: Examine numerical stability and computational accuracy of the explicit and implicit Euler methods, and learn how to verify that your solver works as expected. Learn how to arrange computational experiments and how to plot and evaluate information that reveals method performance.

Programming language: The instructions are Python-oriented, since this is what is used in the basic programming courses at Lund University. You can also use MATLAB if you want, but some of the syntax will have to be modified. If you can explain your code clearly to us at the presentation, then e.g. Julia is also fine. But we can only offer help with Python and MATLAB.

1 Python initialization

The code below assumes that the following libraries have been imported:

```
import numpy as np
import scipy as sp
import scipy.linalg
import matplotlib.pyplot as plt
```

2 General idea

The general approach in the following is that we want to separate the data describing the problem from the data and functionality of the numerical method. We also want to be able to easily change the problem or add new methods to test without copying a lot of code. We will therefore set up a general class `IVP` which describes an initial-value problem and a class `Solver` which does the time-stepping when given a `IVP` instance. A specific IVP will be described by a class inheriting from `IVP` and a specific solver will be implemented by inheriting from `Solver`. At the end of this project, you should be able to run the following code to approximate the solution to the linear test equation $\dot{y} = -y$ with $y(0) = 2$, over the time interval $[0, 1]$ using 100 steps of the explicit Euler method.

```
ivp = LTE(t0 = 0, T = 1, y0 = np.array([2]), A = np.array
    ([[-1]]))
solver = ExplicitEuler(ivp, N = 100)
solver.integrate()

# Plot the approximations {u_n} vs. the temporal
# grid points {t_n}:
plt.plot(solver.grid, solver.us)
```

The `np.array([2])` instead of simply `2` and similarly for `A` ensures that the computations will work in the same way regardless of if we have a scalar-valued or vector-valued problem. When A is a matrix and $y(t)$ is a vector, you would instead do e.g.

```
A = np.array([[-1, 1], [1, -3]])
y0 = np.array([1, 2])
ivp = LTE(t0 = 0, T = 1, y0 = y0, A = A)

solver = ExplicitEuler(ivp, N = 100)
solver.integrate()

# Plot the first component of the approximations {u_n}
# vs. the temporal grid points {t_n}:
plt.plot(solver.grid, solver.us[0, :])
```

You will also implement the implicit Euler method in this project and possibly the trapezoidal rule. In Project 1, you will work on higher-order Runge–Kutta methods. The only change to use these instead should be to replace `solver = ExplicitEuler(ivp, N = 100)` by `solver = ImplicitEuler(ivp, N = 100)` or `solver = RungeKutta4(ivp, N = 100)`.

In order to evaluate the accuracy of the methods, we will finally implement a class `Evaluator`, to be used as follows:

```
ivp = LTE(t0 = 0, T = 1, y0 = np.array([2]), A = np.array
    ([[-1]]))

evaluator = Evaluator(ivp, ExplicitEuler)

# Compute the errors over the grid:
[grid, errs] = evaluator.errvstime(N = 100)
# Plot these errors with log-scale on the y-axis:
plt.figure()
plt.semilogy(grid, errs)

# Compute the error at t = T for different h
# corresponding to N = 2^3, 2^4, ..., 2^10:
Ns = 2**np.arange(3, 11)
[hs, errs] = evaluator.errvsh(Ns)
# Plot these errors in a loglog-scale
plt.figure()
plt.loglog(hs, errs)
# Plot a comparison line which is O(h)
plt.loglog(hs, hs * 2 * errs[-1] / hs[-1])
```

3 The test problem

Consider the linear initial value problem

$$\dot{y} = Ay, \qquad y(t_0) = y_0, \tag{1}$$

where A is a square matrix, over the interval $t \in [t_0, T]$. This is an example of the general problem

$$\dot{y} = f(t, y), \qquad y(t_0) = y_0, \tag{2}$$

where $f(t, y) = Ay$. Note that this choice of f does not explicitly depend on t, but other IVPs could do so.

Task 3.1 Create a class `IVP` for the general problem which has a method `f` with parameters `t` and `y`. This method should not be used directly, it's only there to indicate that any class inheriting from `IVP` should implement this. Thus, have it raise a `NotImplementedError` exception[1]. Also give the class three parameters `t0`, `T` and `y0`, and an initialization method `__init__` that sets these to what the user (you) provides.

[1] This could be done in a more "proper" and sophisticated way by using the abstract base classes provided in the `abc` module.

Task 3.2 Create a class `LTE` (**L**inear **T**est **E**quation) for the test problem (1) which inherits from `IVP`. It needs an initialization method with parameters `t0`, `T`, `y0` and `A` which sets class attributes with the same names. The first three are the general ones defined already in `IVP`, so do

```
def __init__(self, t0, T, y0, A)
  super().__init__(t0, T, y0)
  self.A = A
```

to reuse the general functionality that you already implemented in the parent class `IVP`. It should also have the method

```
def f(self, t, y):
  return self.A @ y
```

which returns the right-hand-side of (1). The `@` is the modern syntax for doing matrix-vector multiplication with NumPy arrays. It also works for matrix-matrix multiplication. Note that this redefinition of `f` "overwrites" the method `f` of the base-class IVP.

Task 3.3 Write down the exact solution to the test problem using the matrix exponential e^{tA}. Then add a method `exact(self, t)` to `LTE` which returns this. Don't do `exp(t*self.A)`, this just exponentiates each element of tA. The matrix exponential method is found in `scipy.linalg.expm`. Check that your method returns the correct answer by testing it with a non-trivial A for which you know what e^{tA} is. *Note:* this function is specific to this test problem! In general, we won't know the exact solution, which is precisely why we need the numerical method.

Task 3.4 Check that you get the expected result when you do e.g.

```
ts = np.linspace(0,1)
ys = [ivp.exact(t) for t in ts]
plt.plot(ts, ys)
```

4 The explicit Euler method

A discretization of the general IVP $\dot{y} = f(t, y)$, $y(0) = y_0$, on the time interval $t \in [0, T]$ is given by the temporal grid points t_n, $n = 1, \ldots, N$ and the approximations $u_n \approx y(t_n)$. Let's use an equidistant grid so that $t_n = nh$, with $h = 1/N$. The explicit Euler method for generating the approximations is given by the formula

$$u_{n+1} = u_n + hf(t_n, u_n) \tag{3}$$

and $u_0 = y_0$. This is an example of a general one-step method,

$$u_{n+1} = u_n + h\Phi(f, h, t_n, u_n) \tag{4}$$

where the step-function Φ is chosen to be $\Phi(f, h, t_n, u_n) = f(t_n, u_n)$.

Task 4.1 Create a class `Solver` for the general one-step method, which has a method `step` with parameters `tn` and `un`. This method should not be used directly, it's only there to indicate that any class inheriting from `Solver` should implement this. Thus it can return anything, or raise an Exception. Also give the class five attributes: `ivp`, `N`, `h`, `grid` and `u`. Initialize `self.ivp`, `self.u` and `self.N` to the user-supplied `IVP` instance `ivp`, y_0 and N, respectively. Then do

```
self.grid = np.linspace(ivp.t0, ivp.T, N + 1)
self.h = (ivp.T - ivp.t0) / N
```

to set up the grid. We don't technically need to store both N and h, but we will use both often and there is no point in repeatedly calculating one of them.

Task 4.2 Create a class `ExplicitEuler` for the explicit Euler method, which inherits from `Solver`. Its `__init__` method should take the same parameters as the Solver `__init__` function and then just call the latter. Then define the `step` method as

```
def step(self, tn, un):
  return un + self.h * self.ivp.f(tn, un)
```

Now our method class knows how to take one step. Next, we need to do this N times. If we only care about the end result $u_N \approx y(T)$ then we could simply do

```
for i in range(self.N):
  self.u = self.step(self.grid[i], self.u)
```

and repeatedly overwrite the approximation `u`. (Recall that we initialized `self.u` to y_0).

However, we are often interested in the intermediate points as well, so let us just store all of them. While this becomes prohibitively expensive in large-scale problems[2], it should not be an issue for us here.

[2]Actually, even storing *one* full approximation vector could be too much

Task 4.3 Add a method `integrate` to the base class `Solver` with no parameters, which does the above iteration. In addition, have it create a list `us` in which you store each approximation. When the method is finished, `us` should contain $N + 1$ vectors, because the first should be y_0. For efficiency, instead of a growing list, preallocate an array with $N + 1$ columns and store each approximation in the appropriate column. You can decide whether you want the method to return `grid` and `us`, or if the user is expected to access them themselves via `solver.grid` and `solver.us`

5 First test

At this point, you should be able to run the first code snippet in Section 2.

Task 5.1 Do so, and verify that everything works as it should. In addition to the numerical approximations, you can also plot the exact solution on the grid for comparison by using your `IVP.exact` function as in **Task 3.4**. In the vector-valued case, you need to transform the list into an array before you can select e.g. the first component. Depending on the choices you make, you might also have to ensure that your arrays have the same shape. It usually does not matter to NumPy if your vector has shape `(3,)` or is represented as a column matrix with shape `(3, 1)`, but Matplotlib is upset when it receives one of each type. You fix it by using the `np.reshape` method.

6 Investigating the errors

In general, we will only ever be able to estimate the error of the method. But for the given test problem, we know the exact solution and can thus compute any error exactly. This allows us to verify that the method behaves like it should. The main error which we are interested is the final global error $err_T = u_N - y(T)$, since we can show that $\|err_T\| = \mathcal{O}(h)$ for the explicit and implicit Euler methods, and e.g. $\mathcal{O}(h^2)$ for the trapezoidal rule. We thus want to compute this error for different choices of h.

It is also interesting to see how the global error changes over time, i.e. how big is $\|u_k - y(kh)\|$ for $k = 1, \ldots, N$? In general we expect it to increase with k, since a small error is made in each step. But depending on the properties of the IVP, it could instead stay roughly equal or even decrease.

Finally, it is interesting to consider both *absolute* and *relative* errors. Absolute errors were used above. A relative error is instead $\|u_k - y(kh)\|/\|y(kh)\|$,

i.e. we scale the absolute error with the size of the exact quantity. If e.g. $y(t) \to 0$ as $t \to \infty$ then the fact that $\|u_N - y(T)\|$ is "small" does not provide much information, since $\|y(T)\|$ could be much smaller. By instead considering the relative error we can automatically adjust our expectations to the inherent scale of the problem. However, since the relative error becomes infinite if the exact solution happens to be exactly zero this is not always the best option. Often a combination of absolute and relative errors is used.

In all cases, we measure the sizes of the errors using a *norm* $\|\cdot\|$. This is not the same as the absolute value function `abs` or `numpy.abs` except possibly in the scalar-valued case. Check the documentation to see what these do by using `?` in the terminal. Given a vector v, the method `scipy.linalg.norm` computes the Euclidean norm $\|v\|_2$ by default, and other norms on demand.

Task 6.1 Create a class `Evaluator` initialized with a IVP object and a Solver type. I.e., you want to be able to call

```
ivp = LTE(t0 = 0, T = 1, y0 = 2, A = -1)
evaluator = Evaluator(ivp, ExplicitEuler)
```

Task 6.2 Give the class a method `errvstime` which takes a parameter N and returns an array `errs` containing the values $\|u_k - y(kh)\|$ for $k = 0, 1, \ldots, N$, as well as the corresponding temporal grid $\{kh\}_{k=0}^{N}$.

Task 6.3 Give the class a method `errvsh` which takes a parameter `Ns` containing a list or array of different N-values. For each N, the method needs to create a new Solver object from the specified type and run its `integrate` method, then extract the final error and put it into an array. Return this array, as well as an array with the corresponding grid sizes h. Note that we specify different N rather than different h, because if we did the latter then for some h there might not be an integer such that $Nh = T$. If so, we would either need to take a fractional number of steps, which is not possible, or take one step too little or one too much, in which case we would not be approximating the same solution value.

Task 6.4 Add an option to both of the methods to compute either absolute or relative errors. (Or some combination, if you want.)

7 More tests

Now test your `Evaluator` class by running something similar to the third code snippet in Section 2. First consider `errvsh`, where we have proved during the lectures that the global error should behave like $\mathcal{O}(h)$.

Task 7.1 Start with the scalar case $A = \lambda$ and the `errvsh` method. Does the error behave like $\mathcal{O}(h)$? Check the lecture notes to see how you determine this from the loglog-plot. Make several graphs for different choices of λ, both negative and positive. Every one should lead to $\mathcal{O}(h)$, though the proportionality constant might differ. This is most easily noticed by plotting several curves in the same plot. You might need to experiment with the choice of N's. (The example uses $N = 2^k$ for different k, why is this smart when using a loglog-scale?) When presenting your work in the following projects, remember to use the functions `xlabel`, `ylabel`, `title`, `legend` and `grid` to make your plots interpretable.

Task 7.2 Repeat your tests, but for the `errvstime` instead. Note that the code snippet uses `semilogy` instead of `loglog`. Why is this more reasonable in this situation? Does the error always grow with time? What happens if you consider instead the relative error?

Task 7.3 Repeat the previous two tasks using some matrix A of your on choice. You can choose any matrix dimension you like, but make sure that A doesn't have too large eigenvalues (check with `scipy.linalg.eig` if necessary). A simple test case could be

$$\begin{bmatrix} -1 & 10 \\ 0 & -3 \end{bmatrix}$$

with $y_0 = \begin{bmatrix} 1 & 1 \end{bmatrix}^{\mathrm{T}}$, $t_0 = 0$ and $T = 10$. Do you see any qualitative differences compared to what you saw in the scalar case? Try a few different matrices, of different sizes, and with different elements.

8 The implicit Euler method

With the code infrastructure you have built, it should now be simple to also add a class for the implicit Euler method, given by

$$u_{n+1} = u_n + hf(t_{n+1}, u_{n+1}). \tag{5}$$

You just need to copy the code for the `ExplicitEuler` class and edit the `step` method. To do this properly, we would need to solve a nonlinear equation since the method is implicit. However, since we will only consider linear problems for now, we can also "cheat" by making use of the linearity.

Task 8.1 When $f(t, y) = Ay$, what is the linear equation system that we need to solve to evaluate u_{n+1} in (5)?

Task 8.2 Create a class `ImplicitEuler` for the implicit Euler method, which inherits from `Solver` just like your `ExplicitEuler` method. Define its `step` method such that it returns the solution to the linear system. Just access `self.ivp.A` which we assume will be there, instead of `self.ivp.f`.

Task 8.3 If you are ambitious, you can ignore the above two tasks above and instead try to find out how to do this using the functions in the root finding part of `scipy.optimize`.

Task 8.4 Run the same tests on your new method as you did for the explicit Euler. In particular, try the matrix

$$\begin{bmatrix} -1 & 100 \\ 0 & -30 \end{bmatrix}$$

with $y_0 = \begin{bmatrix} 1 & 1 \end{bmatrix}^{\mathrm{T}}$, $t_0 = 0$ and $T = 10$. Apply both the explicit and implicit Euler methods. For the plot of the error as a function of time, use $N = 100$ and $N = 1000$. Do both methods have the same stability characteristics? What are the major differences in your observations, and can you give a full explanation?

9 The trapezoidal rule

Task 9.1 If time permits, add a class for analyzing the trapezoidal rule, given by

$$u_{n+1} = u_n + \frac{h}{2}\Big(f(t_n, u_n) + f(t_{n+1}, u_{n+1})\Big). \tag{6}$$

Like for the implicit Euler, you can assume that the problem will be linear.

Task 9.2 See how the method behaves. In particular, study the error versus h and verify that the trapezoidal rule is a *second-order* convergent method.

Task 9.3 Make an error plot where you solve the same linear differential equation using both the implicit Euler method and the Trapezoidal Rule, and compare the accuracy. How much more accuracy do you obtain using the trapezoidal rule?